

MÓDULO: TEMA 4
ARQUITECTURA DE MICROSERVICIOS Y SISTEMAS DISTRIBUIDOS



RETO PRÁCTICO #4

Modernización del Core Bancario Modular

Caso de Estudio	FinBank — Core Bancario Modular	Valor	100 Pts
Punto de partida	Monolito Modular bancario — Java (Spring Boot 3) y .NET 10 (ASP.NET Core Minimal APIs)		
Tecnologías objetivo	Agnósticas — el equipo justifica cada decisión en el ADR		

OBJETIVO DEL RETO

Descripción General

El equipo recibirá un Monolito Modular bancario ya estructurado con límites lógicos claros entre sus dominios. A partir de él, deberá aplicar el patrón Strangler Fig para extraer progresivamente módulos hacia microservicios independientes, establecer comunicación asíncrona entre ellos, e implementar observabilidad y trazabilidad distribuida en toda la arquitectura resultante, incluyendo el monolito remanente. Todas las decisiones de arquitectura tomadas deben quedar formalizadas en un documento ADR. Como punto extra, se implementará una capa de microfrontends.

CONTEXTO DEL NEGOCIO

FinBank es un banco digital cuyo sistema central opera como un Monolito Modular. El sistema ya tiene fronteras lógicas bien definidas entre sus dominios de negocio: cada módulo posee su propio schema de base de datos y expone sus capacidades exclusivamente a través de interfaces internas formales. La organización ha decidido iniciar una modernización progresiva hacia microservicios, priorizando estabilidad del servicio y trazabilidad regulatoria durante la transición.

El equipo recibirá el repositorio del monolito en la tecnología que haya elegido para su implementación:

Aspecto	 Java / Spring Boot 3	 .NET 10 / ASP.NET Core
Framework	Spring Boot 3.2.5, Spring Data JPA	ASP.NET Core 10 Minimal APIs, EF Core

Migraciones DB	Flyway (scripts SQL por schema)	EF Core Migrations (un DbContext por módulo)
Auth	JWT (auth0/java-jwt)	JWT Bearer (ASP.NET Core Authentication)
Capas por módulo	domain / application / infrastructure / api	Domain / Application / Infrastructure / Api
Persistencia	Único datasource, schemas separados en PostgreSQL	Schemas separados, DbContext propio por módulo

Módulos del Monolito Modular

Módulo	Schema DB	Interfaz pública interna	Responsabilidad principal
auth	auth.*	— <i>(solo JWT)</i>	Autenticación de usuarios, emisión y renovación de tokens JWT.
accounts	accounts.*	AccountsService / IAccountsService	Gestión de cuentas bancarias, saldos, débitos y créditos.
transfers	transfers.*	— <i>(orquestador, sin interfaz pública)</i>	Orquestación de transferencias entre cuentas.
notifications	notifications.*	NotificationsService / INotificationsService	Envío de alertas al cliente ante eventos de negocio.
audit	audit.*	AuditService / IAuditService	Registro de todas las operaciones para trazabilidad regulatoria.

Restricción regulatoria del dominio bancario

A diferencia de otros dominios, en banca la consistencia de datos no es negociable para ciertas operaciones. El equipo debe tomar decisiones de arquitectura conscientes sobre cuándo aplica consistencia eventual y cuándo se requiere consistencia fuerte, y documentar cada decisión en el ADR con su justificación.

GUÍA DE EJECUCIÓN PASO A PASO

PASO 1 Primera Extracción con Strangler Fig

Acción

El equipo debe analizar los módulos del monolito, identificar cuál conviene extraer primero según criterios de negocio y arquitectura, y aplicar el patrón Strangler Fig para convertirlo en un microservicio independiente con su propia base de datos exclusiva (Database-per-Service). Se debe implementar un API Gateway o Reverse Proxy que enrute el tráfico correspondiente al nuevo microservicio de forma transparente para el cliente externo, mientras el resto del tráfico continúa hacia el monolito remanente.

Consideraciones de arquitectura

- La elección del módulo a extraer debe justificarse: volumen de tráfico, autonomía del dominio, dependencias entrantes/salientes, riesgo de migración de datos.
- El microservicio debe exponer el mismo contrato HTTP que exponía el módulo en el monolito — los consumidores no deben necesitar cambios.
- Se debe definir y documentar la estrategia de migración de datos del schema del módulo hacia la nueva base de datos exclusiva (sin downtime).
- Los módulos del monolito que dependían del módulo extraído deben adaptarse para invocar el nuevo microservicio de forma remota.

Criterios de selección de tecnología

- **API Gateway / Reverse Proxy:** Libre elección (YARP, Spring Cloud Gateway, Kong, Nginx, Envoy, Traefik, etc.). Justificar en el ADR.
- **Base de datos del microservicio:** Libre elección. El criterio principal es que sea exclusiva del microservicio y adecuada al modelo de datos del dominio. Justificar en el ADR.
- **Contenerización:** Docker + docker-compose o equivalente para el microservicio y su base de datos.

Evidencia requerida

- Justificación documentada de la elección del módulo a extraer.
- Microservicio funcionando autónomamente con su propia base de datos.
- Gateway enrutando el tráfico del módulo extraído al microservicio y el resto al monolito.
- Diagrama de arquitectura actualizado.
- Estrategia de migración de datos documentada.

PASO 2 Segunda Extracción y Comunicación Asíncrona

Acción

Identificar un segundo módulo del monolito que tenga una relación de dependencia con el microservicio extraído en el Paso 1, y extraerlo como un microservicio independiente con su propia base de datos exclusiva. A diferencia del primer paso, este microservicio debe comunicarse con el

PRÁCTICO #4

anterior de forma asíncrona a través de un message broker, resolviendo mediante eventos la interacción que antes ocurría in-process.

Consideraciones de arquitectura

- La elección del segundo módulo debe justificarse en función de su relación de dependencia con el primero y del impacto en la consistencia de datos.
- El equipo debe decidir y documentar el patrón de consistencia distribuida que resolverá la coordinación entre los dos microservicios: Saga (coreografía u orquestación), consistencia eventual con compensación, u otro patrón justificado.
- Los módulos restantes del monolito que dependían del segundo módulo extraído deben recibir sus notificaciones o resultados de negocio a través del broker, sin necesidad de ser extraídos.

Criterios de selección de tecnología

- **Base de datos del segundo microservicio:** Puede ser diferente a la del primero si el modelo de datos del dominio lo justifica. Documentar en el ADR.
- **Arquitectura interna:** El equipo debe elegir y justificar la arquitectura interna de cada microservicio: hexagonal, capas, CQRS, u otra. Documentar en el ADR.
- **Patrón de consistencia distribuida:** Justificar la elección frente a las alternativas evaluadas, considerando el contexto bancario. Documentar en el ADR.

Evidencia requerida

- Justificación documentada de la elección del segundo módulo y su relación de dependencia con el primero.
- Segundo microservicio funcionando autónomamente con su propia base de datos.
- Comunicación asíncrona vía broker entre MS1 y MS2 funcionando.
- Diagrama de arquitectura final: Gateway → MS1 / MS2 / Monolito remanente.
- Diagrama o descripción del patrón de consistencia distribuida elegido.

PASO 3 Resiliencia y Contratos de Eventos**Acción**

Con la comunicación asíncrona entre microservicios establecida en el Paso 2, este paso se enfoca en garantizar la confiabilidad del sistema ante fallos: definir formalmente los contratos de los eventos, asegurar que los módulos del monolito remanente consuman correctamente los eventos que les corresponden, e implementar los patrones de resiliencia necesarios en el contexto bancario.

Consideraciones de arquitectura

- El equipo debe definir qué eventos se publican, quién los produce y quién los consume.
- Los contratos de los eventos (schema, formato, versionamiento) deben quedar documentados.
- Se deben implementar los patrones de resiliencia necesarios para garantizar la confiabilidad del sistema en el contexto bancario.

Patrones de resiliencia — implementar al menos tres

Patrón	Propósito	Qué previene
Circuit Breaker	Cortar el flujo de llamadas cuando un servicio falla repetidamente.	Fallos en cascada y agotamiento de recursos.
Retry + Backoff Exponencial	Reintentar operaciones fallidas con espera creciente.	Fallos transitorios de red o sobrecarga momentánea.
Dead Letter Queue (DLQ)	Aislar mensajes que no pueden procesarse después de N intentos.	Bloqueo de colas y pérdida de mensajes no procesables.
Idempotency Key	Garantizar que procesar el mismo mensaje dos veces no produce efectos dobles.	Dobles débitos o dobles créditos ante reintentos.
Outbox Pattern	Publicar eventos de forma atómica con el commit de la base de datos.	Pérdida de eventos si el broker no está disponible al hacer commit.
Fallback / Degraded Mode	Retornar una respuesta controlada cuando el servicio dependiente falla.	Timeouts colgados y experiencia de usuario degradada sin control.

Criterios de selección de tecnología

- **Message Broker:** Libre elección (Kafka, RabbitMQ, Azure Service Bus, AWS SQS/SNS, NATS, etc.). Justificar en el ADR considerando las necesidades de durabilidad, orden y replay del dominio bancario.
- **Contrato de eventos:** Libre elección de formato y mecanismo de versionamiento (CloudEvents, Avro + Schema Registry, Protobuf, JSON Schema). Documentar en el ADR.

Evidencia requerida

- Arquitectura de eventos documentada: qué eventos se publican, quién los produce y quién los consume.
- Diagrama de secuencia del flujo completo entre los dos microservicios (happy path y failure path).
- Demostración funcional de al menos tres patrones de resiliencia.
- Los módulos del monolito remanente que deben reaccionar a eventos lo hacen correctamente vía broker.

PASO 4 Observabilidad y Trazabilidad Distribuida

Acción

Implementar los tres pilares de observabilidad en todos los componentes de la arquitectura: el API Gateway, los dos microservicios y el monolito remanente. El Trace ID debe propagarse a través de todas las capas — incluyendo los mensajes del broker — de forma que sea posible reconstruir el recorrido completo de cualquier operación de extremo a extremo.

Los tres pilares de la observabilidad

Logs estructurados: formato JSON con campos estandarizados incluyendo traceId, spanId y campos de contexto de negocio relevantes (ej. identificadores de cuenta u operación). Deben ser consultables y correlacionables.

Métricas: indicadores de salud del sistema (latencia P99, tasa de errores HTTP, throughput) y del broker (consumer lag por módulo). Deben permitir generar alertas.

Trazas distribuidas: un único TraceId debe correlacionar los spans de todos los servicios que participan en una misma operación, incluyendo los consumidores del broker en el monolito remanente.

Criterios de selección de tecnología

- **Instrumentación:** Libre elección (OpenTelemetry recomendado por ser agnóstico y estándar de la industria, Micrometer, SDK nativo del proveedor cloud). Justificar en el ADR.
- **Backend de trazas:** Jaeger, Zipkin, Grafana Tempo, AWS X-Ray, Datadog APM, o equivalente.
- **Backend de métricas:** Prometheus + Grafana, Datadog, CloudWatch, o equivalente.
- **Backend de logs:** ELK Stack, Loki + Grafana, Splunk, o equivalente.
- **Propagación de contexto:** El TraceId debe propagarse en headers HTTP (estándar W3C TraceContext recomendado) y en los headers de los mensajes del broker.

Evidencia requerida

- Trace completo de una operación mostrando todos los spans a través de los componentes (gateway, microservicios, módulos del monolito vía broker).
- El mismo TraceId aparece en los logs de todos los componentes para la misma operación.
- Dashboard de métricas con al menos: latencia P99 por servicio, tasa de errores y consumer lag del broker.

PASO 5 Documentación de Decisiones de Arquitectura (ADR)

Acción

Elaborar un documento de Architectural Decision Records (ADR) que formalice todas las decisiones técnicas tomadas durante el reto. Este documento es el entregable más importante desde la perspectiva de un arquitecto de software: evidencia el razonamiento, las alternativas evaluadas y los trade-offs aceptados.

Formato por cada ADR

Título: ADR-XXX — [nombre de la decisión]

Estado: Aprobado / En revisión / Deprecado

Contexto: ¿Cuál era el problema o la necesidad que motivó esta decisión?

Opciones evaluadas: Alternativas consideradas con ventajas y desventajas.

Decisión: ¿Qué se eligió y por qué?

Consecuencias: Implicaciones positivas y negativas, incluyendo deuda técnica introducida.

Decisiones que deben estar documentadas

- Elección del módulo a extraer primero y justificación: (ADR-001)
- Elección del segundo módulo a extraer y justificación: (ADR-002)
- API Gateway / Reverse Proxy elegido: (ADR-003)
- Message Broker elegido: (ADR-004)
- Base de datos del primer microservicio: (ADR-005)
- Base de datos del segundo microservicio: (ADR-006)
- Patrón de consistencia distribuida para operaciones que antes eran atómicas: (ADR-007)
- Arquitectura interna de cada microservicio (hexagonal, capas, CQRS, etc.): (ADR-008)
- Stack de observabilidad: instrumentación, trazas, métricas y logs: (ADR-009)
- Contrato de eventos: formato y versionamiento: (ADR-010)
- Estrategia de migración de datos de los schemas sin downtime: (ADR-011)

Análisis de Trade-offs

El documento ADR debe incluir una sección de análisis de trade-offs que responda las siguientes preguntas:

- ¿La consistencia eventual introducida por la comunicación asíncrona es aceptable para todas las operaciones bancarias de FinBank, o existen operaciones donde no debería aplicarse?
- ¿En qué operaciones fue necesario sacrificar disponibilidad para mantener consistencia?
- ¿Qué tan difícil sería revertir alguna de las decisiones tomadas? ¿Cuáles son reversibles y cuáles no?
- ¿El incremento en complejidad operativa está justificado para el tamaño y carga actuales de FinBank?
- ¿Hubo módulos que consideraron no extraer? ¿Por qué?

Evidencia requerida

- Documento ADR (Markdown, Word o PDF) con mínimo 11 decisiones en el formato especificado.
- Análisis de trade-offs con las preguntas anteriores respondidas de forma argumentada.

★ PUNTO EXTRA: Microfrontends

¿Qué se pide?

Implementar una capa de microfrontends para el portal bancario de FinBank, de modo que el frontend refleje la misma separación de dominios del backend. Cada microservicio debe tener su propio microfrontend independiente, desplegable de forma autónoma y compuesto en tiempo de ejecución por un App Shell.

Arquitectura esperada

Componente	Responsabilidad	Opciones técnicas
App Shell	Routing global, token JWT bancario compartido entre MFEs, composición dinámica en tiempo de ejecución.	<i>Module Federation (Webpack 5), Single-SPA, Piral.</i>
MFE Módulo 1	Interfaz del primer dominio extraído. Consume el microservicio correspondiente.	<i>React, Vue, Angular, Svelte o Web Components.</i>
MFE Módulo 2	Interfaz del segundo dominio extraído. Consume el segundo microservicio. Muestra estado de operaciones en tiempo real (WebSocket / SSE).	<i>Puede ser un framework diferente al del Shell o al del otro MFE.</i>

Criterios de evaluación del punto extra

- Los MFEs se despliegan de forma independiente sin afectar al Shell ni al otro MFE.
- El App Shell carga los MFEs dinámicamente en tiempo de ejecución (no en compilación).
- El token JWT bancario se comparte correctamente entre el Shell y los MFEs sin exponer credenciales.
- Al menos uno de los MFEs muestra el estado de una operación actualizándose en tiempo real.
- La estrategia de composición elegida (Module Federation, iframes, Web Components, Single-SPA) está documentada en el ADR con sus implicaciones de seguridad y despliegue.

CRITERIOS DE EVALUACIÓN

Criterio	Pts	Evidencia clave
Primera extracción con Strangler Fig: microservicio autónomo, gateway, DB exclusiva, migración de datos (Paso 1)	15	<i>Config. gateway + docker-compose + justificación de la elección</i>

PRÁCTICO #4

Segunda extracción: microservicio autónomo con DB exclusiva, comunicación asíncrona con MS1 y patrón de consistencia distribuida (Paso 2)	20	<i>Diagrama de arquitectura + flujo de eventos MS1 ↔ MS2</i>
Resiliencia y contratos de eventos: mínimo 3 patrones demostrados + módulos del monolito consumiendo eventos (Paso 3)	30	<i>Diagrama de secuencia + demos de cada patrón</i>
Observabilidad completa: trace E2E, métricas y logs con Traceld propagado en todos los componentes (Paso 4)	20	<i>Screenshot de trace + dashboard de métricas</i>
ADR con mínimo 11 decisiones + análisis de trade-offs (Paso 5)	15	<i>Documento ADR completo</i>
★ Microfrontends: Shell + 2 MFEs desplegables independientemente + estado en tiempo real	20	<i>Demo de despliegue independiente</i>

Puntaje base: 100 puntos. El punto extra de microfrontends suma hasta 20 puntos adicionales. Los criterios de evaluación aplican independientemente del stack tecnológico elegido.